UNIVERSITY of INFORMATION TECHNOLOGY and MANAGEMENT
in Rzeszow, POLAND

FACULTY OF INFORMATION TECHNOLOGY

Field of Study: INFORMATION TECHNOLOGY
Specialty: Programming

Duy Khanh Mac
No. of student's record book 72850

Real-Time Vision-Language-Action (VLA) Inference for Reactive Robotics

Supervisor: prof. WSIIZ dr inż. Atsuo Murata

DIPLOMA THESIS AT FIRST-CYCLE STUDIES

Rzeszow 2026

Table of Contents

List of Symbols and Abbreviations

The main symbols and abbreviations used in this thesis are listed below.

Symbol / Abbrev.	Meaning
Δ	Inference delay, in control steps
δ	Random temporal offset used during training
ε	Prediction–execution misalignment
s_t , o_t	Robot state and observation at control step t
a_t , A_t	Delta action at step t; action chunk
H , K	Prediction horizon; execution horizon
q	Action-quantization factor
σ	Actuation noise standard deviation (per axis, per step)
e	Delay estimation error, assumed minus true delay
π_θ , v_θ	Policy; flow-matching vector field
I_{pred} , I_{exec}	Prediction interval; execution interval
VLA / VLM	Vision-Language-Action / Vision-Language Model
LoRA / QLoRA	Low-Rank Adaptation / quantized LoRA
RTC / A2C2	Real-Time Chunking / action-chunk correction baseline
SR	Success rate
AdaRMS	Adaptive RMS normalisation (state conditioning)
ODE	Ordinary differential equation
MEM	Multi-Scale Embodied Memory

Introduction

Artificial intelligence now drives much of modern robotics. A robot can read its surroundings and pick a sensible action. AI-driven robots still hesitate, though. Before it acts, the robot waits for its reasoning model to finish, and on tasks that need a quick response that wait is costly. The motion comes out as a “stop–run–stop” pattern that never looks smooth. This thesis asks how a robot can think and move at the same time without losing accuracy, and it studies a new inference framework for Vision-Language-Action (VLA) models that answers the question.

A VLA model is a class of artificial intelligence model built from three parts:

- Vision: images or sensor data from the environment;
- Language: task descriptions in natural language;
- Action: the commands the robot carries out.

A VLA reads its surroundings through vision and the human request through language, then turns both into actions. Most VLAs work by action chunking. At each step the model emits a short sequence of future actions, the robot runs them, and the model emits the next sequence.

Synchronous inference makes the robot wait for the model before it moves, so it stands still while the model thinks. Asynchronous inference lifts that constraint: the robot keeps running the previous actions while the model prepares the next ones, and motion stays almost continuous. The naive version carries a flaw. The robot and the scene keep changing during inference, so the model conditions on one state while the actions land on a later one, and the prediction no longer fits.

The cost shows up in the demonstration videos that accompany VLA papers. Many are sped up several times over so the motion looks fluent; at real speed the robot pauses between chunks and reacts late to anything that moves. Smooth, reactive control at native speed is the practical target, and asynchronous inference promises it as long as the misalignment can be handled.

This thesis studies future-state-aware asynchronous inference, which estimates the robot state at the moment the new actions will run. The VLASH framework [1] is the concrete realisation examined here.

The objectives are:

- to review VLA models and real-time inference;
- to analyse the prediction–execution misalignment that arises in asynchronous inference;
- to explain the future-state-aware method;
- to build a simulation that compares three inference mechanisms: synchronous, naive asynchronous, and future-state-aware;
- to evaluate the simulation results and weigh them against the large-scale results reported for VLASH.

Four chapters follow. Chapter 1 covers the background and related work on VLA models and real-time robotic inference. Chapter 2 sets out the methodology and the design of the simulation together with the future-state-aware method. Chapter 3 reports the experimental setup, the simulation results, and a comparison with the published large-scale results. Chapter 4 draws conclusions, states the contributions, and points to future work.

CHAPTER 1: Background and Related Work

1.0 Introduction to the Chapter

Robotics puts machine-learning models under pressures that offline computer-vision or language tasks never face. Control runs in a loop with a physical or simulated world, in real time, and every millisecond the robot spends thinking is a millisecond in which the world moves on without it. This chapter lays out the background needed to understand that pressure and the methods that address it.

Section 1.1 covers real-time robotic control: control loops, control frequencies, and action chunking. Section 1.2 turns to Vision-Language Models and their extension into VLA models, with a close look at the π0.5 model used here. Section 1.3 formalises inference latency, Section 1.4 reviews the existing asynchronous strategies, and Section 1.5 names the gap that this thesis sets out to close.

1.1 Robotics and Real-Time Control

A robot runs a continuous feedback loop: it senses the environment, computes a response, and executes the result, over and over. The loop has to run fast enough that the world has not changed much between sensing and acting. Manipulation that needs moderate precision runs near 10 Hz, while highly dynamic tasks run above 50 Hz [7, 8].

Early controllers were hand-designed. An engineer wrote down a mathematical model of the robot and its environment and derived the optimal action from it. That demands deep domain knowledge and rarely transfers to messy, unstructured tasks. Deep learning changed the approach: a neural network πθ now maps an observation o_t and a robot state s_t to a sequence of actions,

$$\pi\theta \, (A\_t \mid o\_t \, , \, s\_t \,)$$

Here A_t is the predicted action sequence, o_t the current observation such as RGB camera images, and s_t the proprioceptive state, for example the joint positions and gripper opening.

Action chunking [8] is a key step. Rather than one action at a time, the policy emits a chunk of H consecutive actions in a single forward pass,

$$A\_t = \{ \, a\_t \, , \, a\_\{t+1\} \, , \, \ldots \, , \, a\_\{t+H-1\} \, \}$$

Here H is the prediction horizon. Only the first K actions, the execution horizon, reach the actuators before the policy is queried again. Chunking cuts how often the model runs and smooths out short-term inconsistencies between successive predictions, which raises task completion rates [8].

Chunking creates its own bottleneck under synchronous deployment. The next chunk cannot start until the model finishes a full forward pass on the current observation, and that pass takes tens to hundreds of milliseconds, an inference latency of Δ control steps, during which the robot is frozen. Bigger models stall for longer. On an NVIDIA RTX 4090, π0.5 needs about 103 ms per pass, more than two control steps at 20 Hz [1].

The frequency requirement is strict because errors compound. A controller at 20 Hz has 50 ms to sense, decide, and act; if a decision lands 100 ms late, the robot has already drifted two steps past the state the decision assumed. In free motion that drift is a small wobble, but in a grasp or a contact it can mean a missed object or a dropped one. Higher control rates shrink the per-step error and leave even less time for the model, which is the squeeze this thesis works within.

The stall slows the robot, and it also blinds it: a fresh observation cannot influence motion until the current pass ends. Removing this stall is the motivation behind the thesis.

1.2 Vision-Language Models and VLAs

Large language models gave multimodal learning its footing. A Vision-Language Model (VLM) bolts a visual encoder onto a language model, usually a Transformer image encoder such as SigLIP [14]. The encoder turns an image into tokens, projects them into the text embedding space, and the language backbone then processes image and text together. Trained on large image-text collections, VLMs learn broad visual grounding. PaLI, LLaVA, and PaliGemma [13] are representative.

VLMs perceive and reason well, yet they cannot produce low-level robot commands. Vision-Language-Action models (VLAs) close that gap with an action expert placed on top of a VLM backbone. The VLM reads the current image or images and the task description and forms a context representation; the action expert reads that context together with the robot state and outputs an action chunk. The whole stack trains end to end from human demonstrations.

RT-2 [12] was among the first to fine-tune a web-pretrained VLM for manipulation. π0 [2] added a flow-matching action expert and reached high dexterity on contact-rich tasks. GR00T N1 [15] scaled the idea to humanoid robots. This thesis uses π0.5 [3], a refinement aimed at following instructions across varied manipulation tasks. Table 1 places these models side by side.

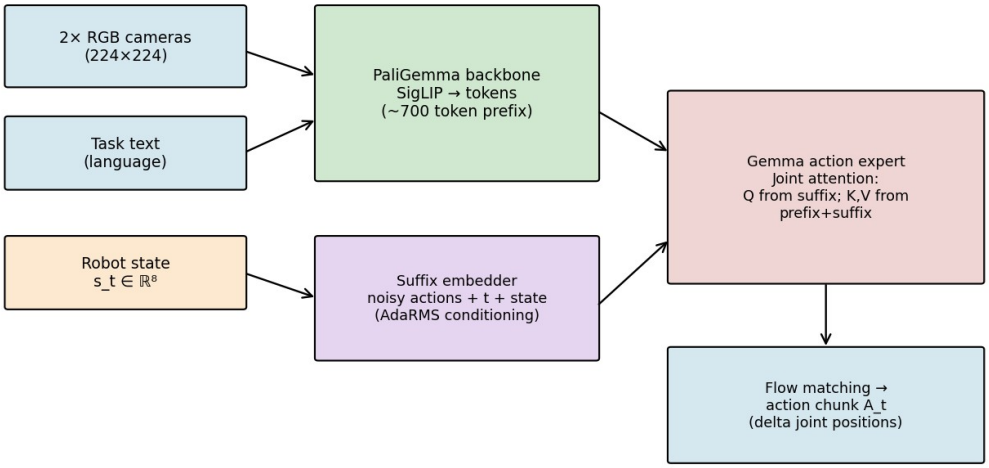
Tab. 1. Representative Vision-Language-Action models.

Model	Vision-language backbone	Action representation	Year
RT-2 [12]	Web-pretrained VLM (PaLI-X)	Discrete action tokens	2023
π0 [2]	PaliGemma	Flow matching (continuous)	2024
π0.5 [3]	PaliGemma + Gemma expert	Flow matching (continuous)	2025
SmolVLA [4]	Compact VLM	Flow matching (continuous)	2025
GR00T N1 [15]	Eagle VLM	Diffusion (continuous)	2025
OpenVLA [16]	Prismatic VLM	Discrete action tokens	2024

Source: compiled from [2, 3, 4, 12, 15, 16]

Figure 1 shows the π0.5 architecture. Its vision-language backbone is PaliGemma, which encodes up to two camera images with a SigLIP encoder and processes the patch tokens next to the tokenised instruction. A separate Gemma model, the action expert, produces the action tokens. Joint attention [2] links the two: at every layer the action expert's queries attend over its own keys and values and over the backbone's, so the action expert sees the full visual and linguistic context without slowing inference.

Fig. 1. Architecture of the π0.5 vision-language-action model: a PaliGemma backbone and a Gemma action expert coupled by joint attention, with state conditioning and a flow-matching action head.



Source: own work, based on [1, 3]

π0.5 feeds the robot state in through AdaRMS, an adaptive root-mean-square normalisation whose scale and shift come from a linear projection of the state and a diffusion timestep embedding. Earlier VLAs chopped actions into discrete tokens; π0.5 instead generates continuous actions with flow matching [11], learning a vector field vθ(x, t) that defines a probability-flow ODE,

$$dx / dt = v\theta\ (\ x\ ,\ t\ )$$

Integrating the ODE from t = 0 to t = 1 carries a noise sample to a predicted action chunk. Flow matching follows straighter paths than diffusion, so ten integration steps are often enough, which matters when the latency budget is tight.

The field is trained by regressing it onto the straight-line velocity between noise and data. For a noise sample x_0 and a target action chunk x_1, the point at time t is x_t = (1 − t) x_0 + t x_1, and the target velocity is the constant x_1 − x_0. Training minimises the mean-squared error between the predicted and the target velocity,

$$L = E_{t, x_0, x_1}\ \| v_\theta(x_t, t) - (x_1 - x_0) \|^2$$

Because the target path is a straight line, the learned ODE is smooth and quick to integrate, which is what lets a handful of steps suffice at inference. To fit a new task, the pretrained π0.5 weights are adapted with Low-Rank Adaptation (LoRA) [10], which updates under 1% of the parameters.

1.3 Action Chunking and Inference Delay

An action-chunking policy emits H actions per query (Section 1.1). Two intervals make the timing precise. At decision time t the policy reads o_t and s_t and returns a chunk that covers the prediction interval I_pred = [t, t + K), where K ≤ H is the execution horizon. Under synchronous control the forward pass starts at t and finishes Δ control steps later, with the robot halted throughout, so the first K actions really run over the execution interval I_exec = [t + Δ, t + Δ + K).

The two intervals expose the prediction–execution misalignment. The policy conditioned on the state at t, but the actions do not start until t + Δ, and by then the robot sits in a different state. The larger Δ is, the larger the mismatch. Table 2 lists measured π0.5 latencies on a few hardware platforms [1].

Tab. 2. Measured inference latency of π0.5 on representative hardware platforms.

Hardware	Inference time	Control steps (at 10 Hz)
NVIDIA RTX 4090	~103 ms	~1 step
NVIDIA RTX 5090	~30 ms	< 1 step
Consumer laptop GPU	~200–400 ms	2–4 steps

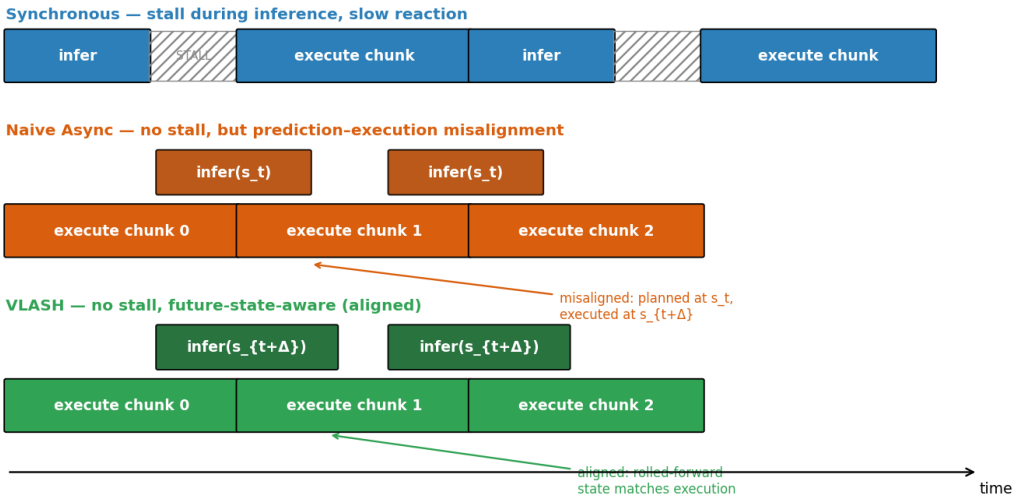
Source: based on [1]

At 10 Hz an RTX 4090 loses about one step, enough to matter for a reactive task, and a laptop GPU loses three or four. Synchronous stalling and misalignment then bite in two ways. The stall lowers the effective control frequency and shows up as visible hesitation. The misalignment makes the policy command a pose the robot has already left, and in contact-rich manipulation those small errors pile up. Larger models score higher but widen both gaps, and that tension is the problem this thesis sets out to solve.

1.4 Asynchronous Inference in Robotics

The stall has a simple fix: run inference on one thread and execution on another, so the model builds the next chunk while the robot runs the current one. If inference finishes before the current chunk runs out, the swap costs no idle time and the robot never stops. Reaction latency then drops from a full Δ-step wait to whatever time is left in the current chunk [1]. Figure 2 sets the three paradigms side by side.

Fig. 2. Three inference paradigms. Synchronous inference stalls the robot; naive asynchronous inference removes the stall but leaves prediction–execution misalignment; VLASH conditions on the rolled-forward future state, removing the misalignment.



Source: own work, based on [1]

Asynchronous inference also asks more of the software than synchronous inference does. The model has to run on a background thread or process while the control loop keeps stepping on the main one, and the finished chunk has to cross between them without a lock that would bring the stall back. On a workstation this is routine. On embedded hardware with few cores and a real-time scheduler it is a real source of engineering risk, and the methods below differ in how much of this machinery they add on top of the basic thread split.

Naive asynchronous inference, the version in SmolVLA [4], queries the policy with the observation and state taken when inference begins and swaps the new chunk in as soon as it is ready, with no correction for the time that passed. It kills the stall and keeps latency low, yet it leaves the misalignment untouched. Inference begins at t and ends at t + Δ; the policy conditioned on s_t, but

the actions run from s_{t+Δ}. The gap s_{t+Δ} − s_t widens with Δ, and the motion turns jittery and laggy on contact-rich tasks.

Real-Time Chunking (RTC) [5] edits the chunk at runtime. It freezes the actions already executed and regenerates the rest, inpainting the suffix from the frozen prefix. RTC does cut the misalignment, but the inpainting step is an extra forward pass run right when the chunk is consumed, so it adds latency at the worst moment, and it works only if the policy was trained to inpaint.

A2C2 [6] takes a different route. It bolts a small correction module onto the base policy and trains it to predict, from the latest observation, per-step corrections to the pending chunk. The module works with an off-the-shelf VLA and leaves the base weights alone, but it is still a trained add-on that runs every step, and it never models the robot's future pose directly. So the three options each fall short. Naive async ignores the misalignment, RTC fixes it but pays for inpainting at runtime, and A2C2 adds a correction head and a per-step cost. None reaches zero overhead, no extra module, and explicit future-state compensation at once.

1.5 Limitations of Existing Approaches

Each strategy beats synchronous deployment, and each has its own catch. Naive async swaps the stall for a misalignment that grows with Δ, and on precise or dynamic tasks it falls behind synchronous inference even with the latency advantage [1]. RTC fixes the misalignment but puts inpainting on the critical path and needs a policy trained to inpaint. A2C2 trims the error during execution, yet it adds a trained module and a per-step cost, and it works on the action values rather than on the future pose. Every asynchronous method also needs multi-threaded execution. Table 3 sets the options against the four properties an ideal method should have.

Tab. 3. Inference strategies against the four desirable properties.

Property	Sync	Naive	RTC	A2C2	VLASH
No action stall	no	yes	yes	yes	yes
No runtime overhead	yes	yes	no	no	yes
No architecture change / added module	yes	yes	no	no	yes
Misalignment compensated	n/a	no	yes	yes	yes
Models future kinematic state	n/a	no	no	no	yes

Source: own analysis, based on [1, 4, 5, 6]

The gap is now clear. A good solution would meet four conditions at once: (1) no runtime cost beyond the ordinary forward pass; (2) no change to the architecture, so any pretrained checkpoint drops in; (3) explicit misalignment compensation, with the policy conditioned on the state its actions will actually run from; and (4) deployment as a plain fine-tuning step. No current method meets all four. Chapter 2 studies one that does.

1.6 Summary of the Chapter

This chapter set the stage. Real-time control runs on a tight latency budget, and action chunking only postpones the latency problem rather than removing it. VLAs such as π0.5 pair a strong vision-language backbone with continuous, flow-matched actions, but on real hardware their inference opens a gap of one to four control steps between prediction and execution. That gap, the prediction–execution misalignment between I_pred and I_exec, is what really drags accuracy down under

asynchronous deployment, not the stall by itself. Each existing method handles only part of it. Chapter 2 turns to the methodology and the simulation, and to state rollforward, which conditions the policy on a future state computed in closed form at no extra inference cost.

CHAPTER 2: System Design and Methodology

2.0 Introduction to the Chapter

Chapter 1 showed why synchronous VLA inference falls short for reactive robotics, and why no existing asynchronous method delivers zero runtime overhead, architectural compatibility, and accurate misalignment compensation together. This chapter sets out the methodology and the system built to study and test a solution.

The work has two parts. The first studies the VLASH method [1] and lays it out in one place (Section 2.2). The second, the practical core, designs and builds an original simulation testbed (Sections 2.3–2.4) that recreates the prediction–execution misalignment in a controlled, fully reproducible setting and measures how synchronous, naive asynchronous, and future-state-aware inference behave as the delay grows. The testbed drops the heavy VLA model on purpose, so the misalignment shows up on its own and can be measured without a GPU cluster.

Section 2.1 states the goals and scope. Section 2.2 summarises the VLASH method. Section 2.3 describes the testbed: the task, the state and action representation, the inference-delay model, the three controllers, and the metrics. Section 2.4 documents the implementation and tools. Section 2.5 relates the testbed to the full-scale reference implementation and the author's hardware. Section 2.6 summarises the chapter.

2.1 Overview and Objectives of the System

The system is a lightweight, CPU-only simulation testbed that runs the three inference strategies of Chapter 1 on one common task and compares them. The question it answers is practical: as the inference delay Δ grows, how does each strategy lose accuracy, lose smoothness, and build up prediction–execution misalignment?

Three simplifications keep the mechanism in view, each backed by Chapter 1. The robot becomes a point end-effector on a 2-D plane, which keeps the delta-action property that makes rollforward exact and drops the cost of a full kinematic or contact simulator. The policy becomes a deterministic open-loop chunk planner, which keeps the one feature of action chunking that causes the misalignment: a block of actions committed in advance. The inference delay Δ becomes a plain integer number of control steps, so it can be swept as an independent variable. The testbed does not replace the full-scale experiments. It isolates the mechanism and confirms it, and the large-scale numbers back it up in Section 3.4.

One consequence of these choices is worth stating early. The testbed measures the misalignment mechanism, not a particular robot. A result such as a thirty-point gap between VLASH and naive async at a large delay is evidence about the mechanism, and the matching large-scale numbers in Section 3.4 supply the real-robot grounding. Keeping the two roles apart is what lets a laptop-scale study say something credible about a GPU-scale system.

2.2 The VLASH Method Under Study

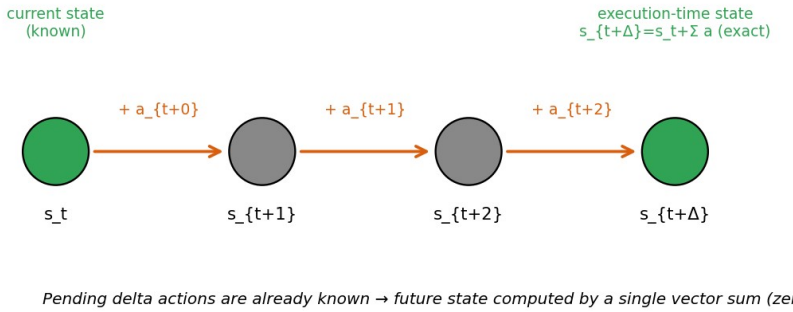
2.2.1 State rollforward

Under asynchronous inference the robot is still running the current chunk when the policy is queried at step t. Actions are delta joint positions (s_{t+1} = s_t + a_t), so the state at which the new chunk will start is just a sum of the pending actions, computed exactly and almost for free:

$$s_{t+\Delta} = s_t + \Sigma_{i=0}^{\Delta-1} a_{t+i}$$

The policy then conditions on this rolled-forward state, $\pi\theta (A_{t+\Delta} | o_t , s_{t+\Delta})$, in place of the stale current one. That swap is the whole runtime change, and it removes the state-misalignment error with no extra model call. Figure 3 shows the computation.

Fig. 3. State rollforward. The pending delta actions are already known, so the execution-time state is obtained exactly by a single vector summation, at zero inference overhead.



Source: own work, based on [1]

2.2.2 Temporal-offset augmentation

Rollforward helps only if the policy has learned to use a state that is offset in time from its observation. VLASH teaches it that during fine-tuning. For each training sample it draws a random offset $\delta \sim \text{Uniform}\{0, \ldots, \text{max\_delay\_steps}\}$ and pairs $(o_t, s_{t+\delta})$ with $a_{t+\delta : t+\delta+H}$. Across the full range of offsets the policy learns to handle any delay, the synchronous case $\delta = 0$ included.

2.2.3 Shared-observation optimisation

Every offset at a given timestep shares the same observation of roughly 700 tokens, so VLASH packs N offset branches into one forward pass behind a block-sparse attention mask. The observation tokens are encoded once and read by all branches, while the branches stay isolated from each other. One observation is about 700 tokens and one state-action branch about 50, so packing five branches grows the sequence by roughly 20% while multiplying the effective training pairs fivefold. That imbalance is where the saving comes from, and the authors report a 3.26× training speed-up at equal effective batch size.

2.2.4 Action quantization

Grouping q consecutive delta actions into a macro-action $\hat{a}_i = a_{iq} + \ldots + a_{(i+1)q-1}$ lets the policy emit H/q outputs per chunk, trading temporal resolution for speed. At $q = 2$ the authors report a 2.03× end-to-end speed-up with little accuracy lost [1]. The testbed concentrates on the one component that sets deployment-time accuracy, state rollforward. It models temporal-offset augmentation only implicitly, through a planner that accepts whatever conditioning state it is handed.

2.3 Design of the Simulation Testbed

2.3.1 The tracking task

The testbed runs a reactive reaching task. A point end-effector at position $s \in \mathbb{R}^2$ has to stay on a moving target $g_t \in \mathbb{R}^2$. The target moves like a staircase: it holds a fixed spot for about 40 control

steps, jumps by a random step of up to 0.25, then holds again. While the target holds, any overshoot from misalignment becomes pure error, which is the point. An early design used a smoothly drifting target, but there the naive overshoot accidentally led the target and hid the effect. The jumps test how fast each strategy reacts.

2.3.2 State and action representation

The action is an incremental displacement $a \in \mathbb{R}^2$, clipped to a maximum size per step, and the dynamics are the exact sum s_{t+1} = s_t + a_t, matching the delta-action property VLASH relies on. This is the smallest representation for which rollforward stays exact.

2.3.3 The chunk planner

From a conditioning state s_cond and an observed target g_obs, the planner builds an open-loop chunk of K identical delta actions that, run from s_cond, carry the end-effector onto g_obs:

$$a_k = clip\ (\ (g\_obs - s\_cond) / K\ ,\ -MAX\_STEP\ ,\ +MAX\_STEP\ )\ ,\ \ k = 0 \ldots K-1$$

The chunk is open-loop: the same fixed deltas apply wherever the robot happens to be. Run the chunk from a different start state and the deltas do not change, but the landing point shifts by exactly the misalignment ε = s_exec_start − s_cond. That makes the planner a clean probe for the mechanism.

2.3.4 The three controllers

All three controllers share the planner and differ only in when inference runs and which state it conditions on. In synchronous inference (Algorithm 1) the robot freezes for Δ steps while inference runs, then executes a chunk planned from the state it just saw; there is no misalignment, but the robot sits idle and reacts slowly. In naive asynchronous inference (Algorithm 2) the robot never freezes. Inference for the next chunk starts Δ steps before the current one runs out and conditions on the stale state s_t, so the misalignment is ε = ‖s_{t+Δ} − s_t‖. In VLASH (Algorithm 3) the conditioning state is rolled forward through the known pending deltas, so it equals the true execution-start state and the modelled misalignment is ε = 0 by construction. In both asynchronous cases the target observation g_obs stays stale, since the simulation cannot see the future scene. That reproduces the stated VLASH limitation: rollforward fixes the robot state, not the visual change. Algorithms 1 to 3 give the three control loops in full.

Algorithm 1 Synchronous control loop

```
Input: policy pi, env, horizon K, delay D
while task not done:
    wait D steps                # robot frozen (action stall)
    o, s <- env.observe()
    A   <- pi(o, s)             # plan an open-loop chunk
    for k = 0 .. K-1:
        env.step(A[k])          # execute the chunk
```

Algorithm 2 Naive asynchronous loop

```
Input: pi, env, K, D ;  A <- pi(env.observe()) ;  k <- 0
while task not done:
    env.step(A[k]) ;  k <- k + 1
    if k = K - D and no inference pending:
        o, s <- env.observe()
        launch A_next <- pi(o, s)        # stale state s_t
    if k >= K:  A <- A_next ;  k <- 0    # swap, no stall
```

Algorithm 3 VLASH (future-state-aware) loop

```
Input: pi, env, K, D ;  A <- pi(env.observe()) ;  k <- 0
```

```
while task not done:
    env.step(A[k]) ;  k <- k + 1
    if k = K - D and no inference pending:
        o, s <- env.observe()
        s_future <- s + sum(A[k .. k+D-1])   # roll state forward
        launch A_next <- pi(o, s_future)     # future-state-aware
    if k >= K:  A <- A_next ;  k <- 0
```

2.3.5 Evaluation metrics

Each episode records four metrics, averaged over the trials. Misalignment ε is the mean ‖s_exec_start − s_assumed‖ at each chunk swap, the quantity that measures prediction–execution misalignment. Tracking error is the mean distance ‖s − g‖ over the episode. Success rate is the share of control steps where ‖s − g‖ stays within a fixed tolerance. Smoothness, or jerk, is the mean change between consecutive actions, smaller being smoother. Reaction latency is handled with the analysis of Chapter 1 rather than measured here, because the kinematic abstraction understates the real synchronous stall and would make the asynchronous reaction advantage look smaller than it is.

2.4 Implementation

The testbed is written in Python 3, with NumPy for the dynamics and metrics and Matplotlib for the figures. It needs no machine-learning framework and no GPU, so a run finishes in seconds on an ordinary laptop. The code is one documented module: a target generator, the chunk planner, the three controllers, the metric functions, and a driver that sweeps the delay and collects the results.

The main parameters are the execution horizon K = 8, the action clip MAX_STEP = 0.05, the target jump interval of about 40 steps, the jump size of 0.25, the success tolerance of 0.03, and the episode length of 600 steps. Each delay $\Delta \in \{0, 1, 2, 3, 4\}$ runs over 50 trials with distinct seeds, and every reported value is the mean across them. A single base seed drives all randomness, so re-running the module reproduces every number exactly; the per-trial results and the summaries are written to CSV next to the figures. The work used Python 3.11 with NumPy, pandas, and Matplotlib, under Git version control in Visual Studio Code.

Two checks guard the implementation. At delay $\Delta = 0$ all three controllers reduce to the same closed-loop planner and return identical metrics, which confirms that they differ only in how they handle the delay. And the misalignment ε measured for VLASH is exactly zero at every delay, as the rollforward construction requires, so any nonzero value would flag a bug. Both checks pass on every run.

2.5 Relation to the Full-Scale Reference Implementation

The testbed is a small stand-in for a much larger system. The full-scale reference, the source of the corroborating numbers in Section 3.4, is the VLASH codebase [1] on the HuggingFace LeRobot framework [19], which fine-tunes π0.5 [3] (a PaliGemma backbone [13], a Gemma action expert, and a flow-matching action head [11]) on the LIBERO benchmark [7]. There the state is the eight-dimensional Franka Panda configuration, the observation is two 224×224 RGB images with a language instruction, and the action is an eight-dimensional delta joint command, the same delta property the testbed leans on.

LIBERO itself is a tabletop manipulation benchmark on a Franka Panda arm, with four sub-benchmarks that each probe a different kind of generalisation: spatial layout, object identity, goal

specification, and long-horizon tasks. The VLASH numbers in Section 3.4 average over all four, so a single accuracy figure stands in for a broad slice of manipulation rather than one narrow task.

The longer-term target is the author's own machine, an NVIDIA RTX 3050 (6 GB) laptop, where the reference setup uses 4-bit QLoRA quantization and LoRA adapters to fit the model in memory. A full fine-tune and LIBERO run on that hardware is the natural next step, set out in Section 4.1. Within the scope of this thesis, the simulation testbed is the practical contribution that is feasible to deliver.

2.6 Summary of the Chapter

This chapter laid out the methodology. It summarised the four parts of VLASH, namely state rollforward, temporal-offset augmentation, the shared-observation optimisation, and action quantization, and it described the original contribution: a lightweight, reproducible simulation testbed that runs a reactive reaching task with delta actions and an explicit delay model, through three controllers that differ only in the state they condition on. Four metrics track behaviour as the delay grows. Chapter 3 reports the testbed results and checks them against the large-scale VLASH evaluation.

CHAPTER 3: Experiments and Results

3.0 Introduction to the Chapter

Chapters 1 and 2 established the motivation for future-state-aware asynchronous inference and the simulation testbed designed to study it. This chapter reports the experimental results in two complementary parts. The first and primary part (Sections 3.1–3.3) presents the results of the original simulation testbed, which isolates and quantifies the prediction–execution misalignment mechanism under controlled conditions. The second part (Section 3.4) corroborates those findings against the large-scale evaluation reported for VLASH [1] on the LIBERO and Kinetix benchmarks and on real robots. Section 3.5 discusses the findings and limitations; Section 3.6 summarises the chapter.

3.1 Experimental Setup

Simulation study. All results in Sections 3.2–3.3 are produced by the testbed of Chapter 2. The inference delay is swept over $\Delta \in \{0, 1, 2, 3, 4\}$ control steps; for each delay, every method is evaluated over 50 independent episodes of 600 control steps with a staircase target that jumps at randomised intervals of roughly 40 steps. The execution horizon is $K = 8$, the per-step action magnitude is capped at 0.05, and the success tolerance is 0.03. All reported values are means over the 50 trials and are exactly reproducible from a single random seed. Two further sweeps at a fixed $\Delta = 3$ vary the actuation noise σ and the delay estimate used by rollforward; Sections 3.3.6 and 3.3.7 define each manipulation where its results are reported.

Large-scale reference. The corroborating results in Section 3.4 are those reported for $\pi 0.5$ [1, 3]. The LIBERO experiments fine-tune $\pi 0.5$ for 30K iterations at batch size 32 with execution horizon $K = 5$; latency is measured on a laptop RTX 4090 at 103 ms per forward pass over two images. The real-world experiments use $H = 50$, $K = 24$ at 30 Hz on Galaxea R1 Lite and LeRobot SO-101 arms; the reaction-speed measurements use $K = 25$ at 50 Hz across RTX 5090, 4090, and 5070 GPUs.

3.2 Evaluation Metrics

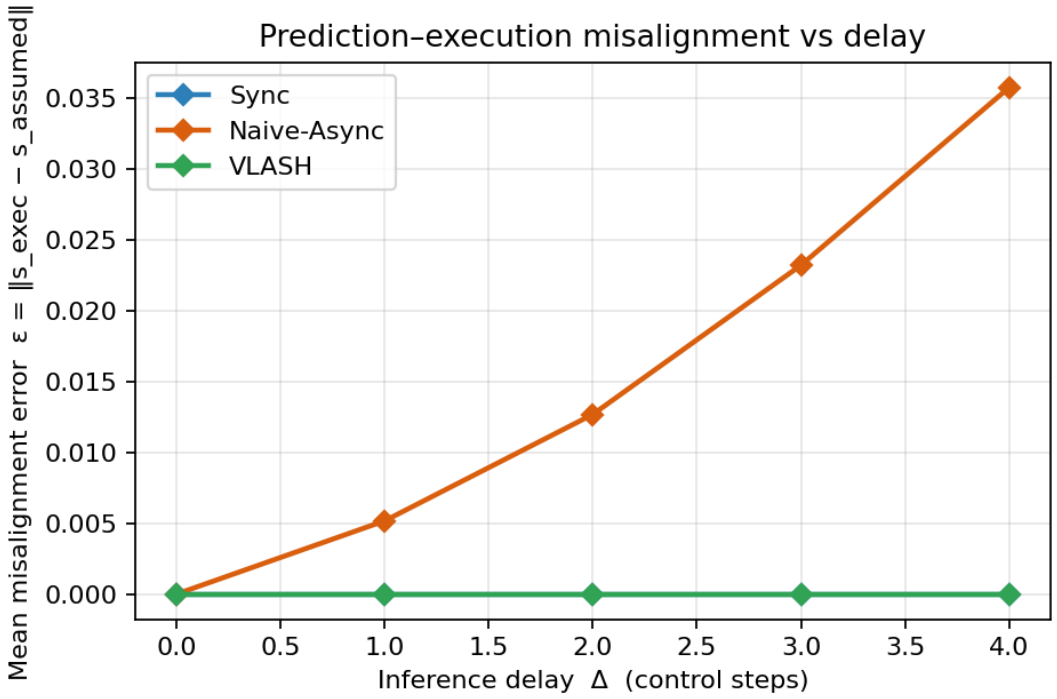
The simulation study reports four metrics, defined in Section 2.3.5 and restated here. Misalignment ε is the mean gap between the state a chunk was conditioned on and the state from which it actually begins executing. Tracking error is the mean distance between the end-effector and the target. Success rate is the fraction of control steps within tolerance of the target. Smoothness is the mean magnitude of change between consecutive actions, smaller being smoother. The large-scale reference results in Section 3.4 additionally use task success rate, completion time, speed-up, and reaction latency as defined by the VLASH authors [1].

3.3 Simulation Study Results

3.3.1 Misalignment grows with delay but is eliminated by rollforward

Figure 4 plots the measured misalignment ε against the delay. For naive asynchronous inference ε climbs with Δ, from 0.005 at $\Delta = 1$ to 0.036 at $\Delta = 4$, because the chunk conditions on a state the robot has already left by the time it runs. For synchronous inference and for VLASH, ε is zero at every delay: synchronous inference never starts from a state other than the one it planned for, and VLASH rolls the state forward so the conditioning state lands exactly on the execution-start state. This is the clearest evidence for the central claim. Table 4 collects all four metrics.

Fig. 4. Prediction–execution misalignment ε versus inference delay. Naive async grows with Δ; VLASH and synchronous inference remain at zero.



Source: own work

Tab. 4. Simulation results (mean over 50 trials per delay).

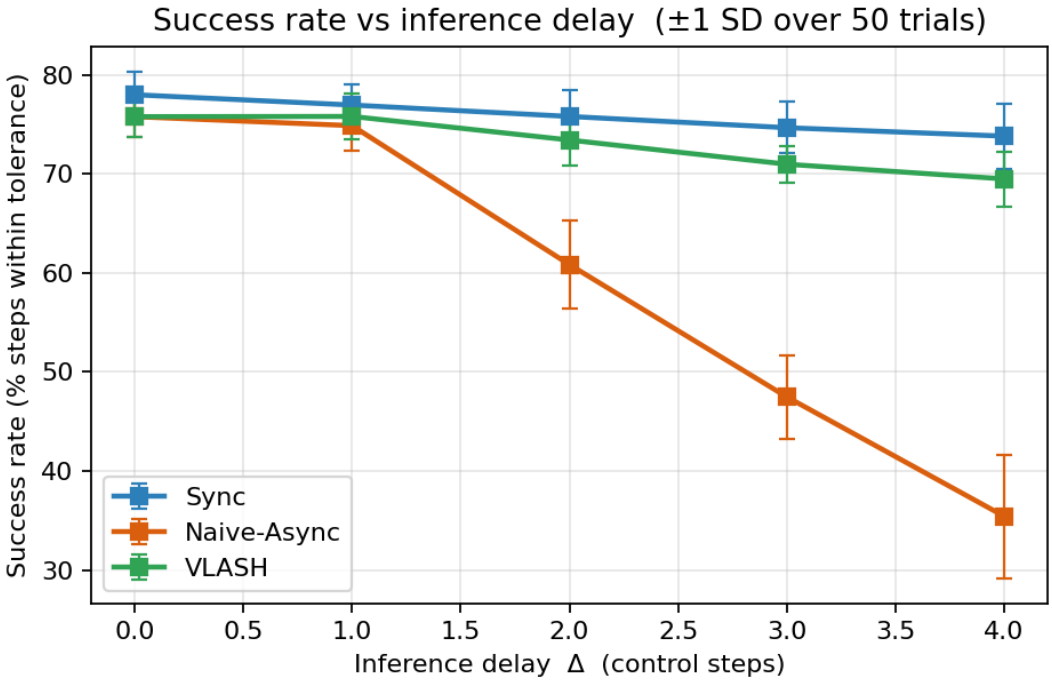
Δ	Method	Misalign. ε	Track. error	Success (%)	Jerk (×10⁻³)
0	Sync	0.000	0.029	78.0	1.02
0	Naive-Async	0.000	0.033	75.8	1.02
0	VLASH	0.000	0.033	75.8	1.02
1	Sync	0.000	0.032	76.9	1.03
1	Naive-Async	0.005	0.038	74.9	1.14
1	VLASH	0.000	0.034	75.8	1.03
2	Sync	0.000	0.034	75.8	1.03
2	Naive-Async	0.013	0.049	60.8	1.21
2	VLASH	0.000	0.038	73.4	1.02
3	Sync	0.000	0.037	74.7	1.07
3	Naive-Async	0.023	0.061	47.5	1.28
3	VLASH	0.000	0.043	71.0	1.06
4	Sync	0.000	0.038	73.8	1.03
4	Naive-Async	0.036	0.071	35.4	1.36
4	VLASH	0.000	0.046	69.5	1.02

Source: own work

3.3.2 Task accuracy

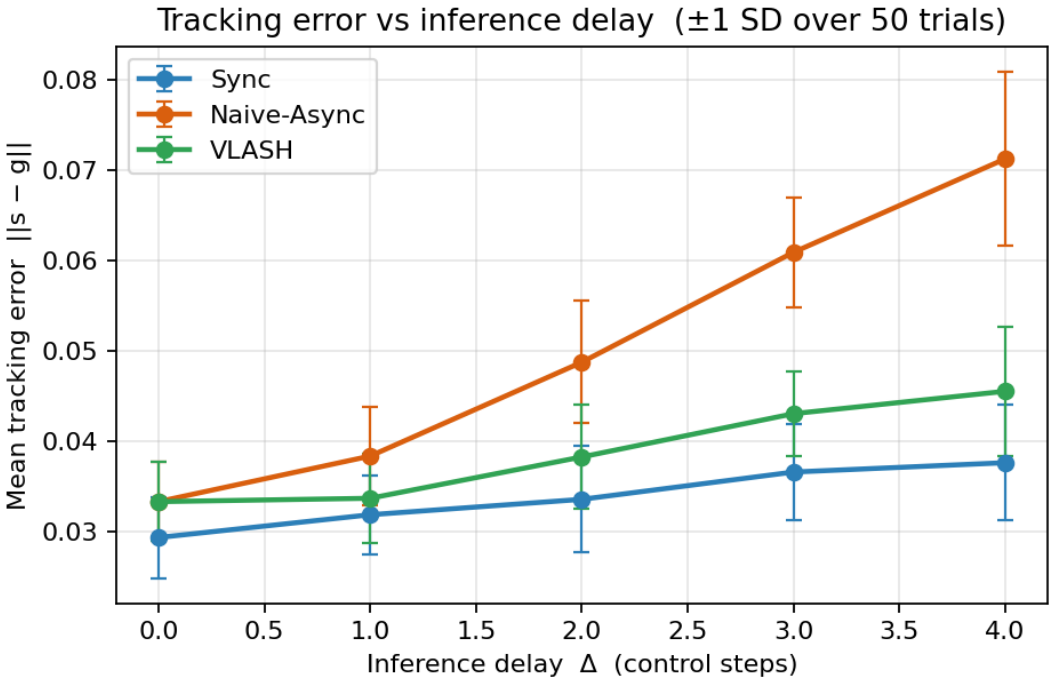
Figures 5 and 6 plot success rate and tracking error against delay. Synchronous inference sets the accuracy ceiling and slips only a little, from 78.0% to 73.8% across the range; that small loss comes from reacting late to target jumps, not from misalignment. Naive asynchronous inference starts at the same level and then collapses, down to 35.4% success at Δ = 4, as the accumulated misalignment carries the open-loop chunk past the target. VLASH stays close to the synchronous curve and holds 69.5% at Δ = 4, about double the naive baseline, while remaining asynchronous. Tracking error says the same: at Δ = 4 it is 0.038 for synchronous, 0.046 for VLASH, and 0.071 for naive async. The error bars in Figures 5 and 6 mark ±1 standard deviation over the 50 trials; from Δ = 2 onward the three methods separate by well more than that spread.

Fig. 5. Success rate versus inference delay. VLASH closely tracks the synchronous upper bound while the naive baseline collapses.



Source: own work

Fig. 6. Mean tracking error versus inference delay.



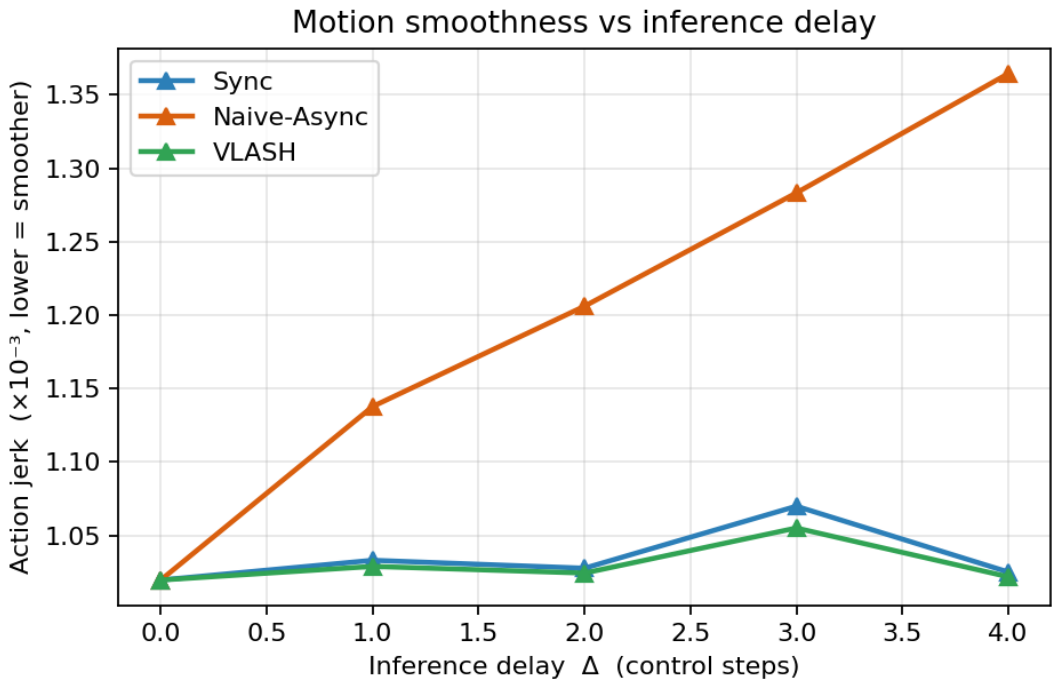
Source: own work

3.3.3 Motion smoothness

Figure 7 plots the action jerk. Naive asynchronous inference grows rougher as the delay rises, with jerk climbing from 1.02 to 1.36 $\times 10^{-3}$, a sign of the corrective jumps that follow each over-shot chunk. VLASH stays as smooth as synchronous inference, around 1.02–1.06 $\times 10^{-3}$, at every delay.

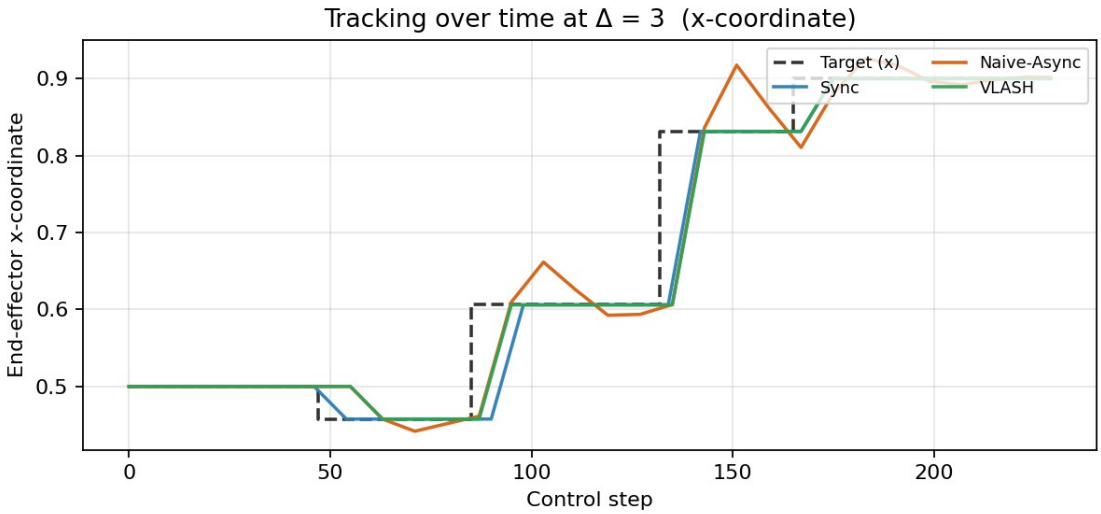
Figure 8 shows the same thing directly: after each target jump the naive trace overshoots the new target and then corrects, while the VLASH and synchronous traces settle onto it cleanly.

Fig. 7. Motion smoothness (action jerk) versus inference delay; lower is smoother.



Source: own work

Fig. 8. End-effector tracking over time at Δ = 3 (x-coordinate). After each target jump the naive trace overshoots the new target before correcting, while VLASH and synchronous inference converge onto it cleanly.



Source: own work

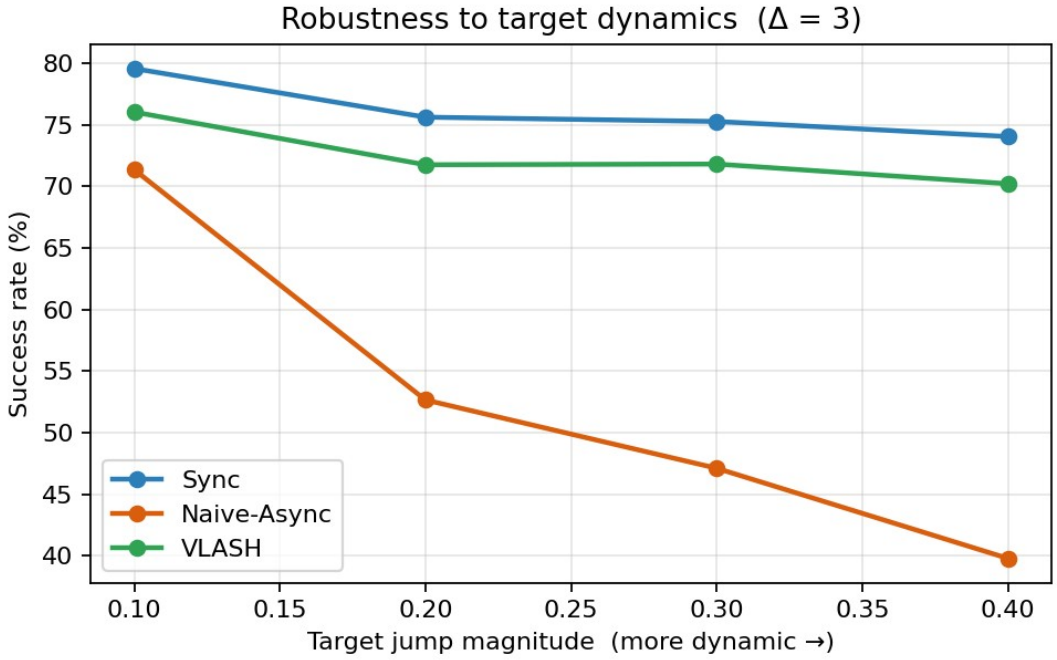
3.3.4 A residual gap to synchronous, explained by stale perception

A small gap stays between VLASH and synchronous inference, 69.5% against 73.8% at Δ = 4. The gap is expected and worth reading. Rollforward fixes the robot state exactly, but every asynchronous method still works from the target observation taken when inference began, so that observation is stale by Δ steps. The testbed reproduces, at small scale, the limitation the VLASH authors name: future-state awareness covers proprioceptive change, not change in the visual scene.

3.3.5 Robustness to target dynamics

The misalignment grows with how fast the scene moves, so the gap between the methods should widen as the task gets more dynamic. To test this, the target jump magnitude was swept from 0.10 to 0.40 at a fixed delay of Δ = 3, with everything else unchanged. Figure 9 and Table 5 report the success rate of each controller.

Fig. 9. Success rate versus target jump magnitude at Δ = 3. As the target becomes more dynamic, naive async falls away while VLASH stays close to the synchronous ceiling.



Source: own work

Tab. 5. Success rate (%) versus target jump magnitude (Δ = 3).

Jump magnitude	**0.10**	**0.20**	**0.30**	**0.40**
Sync	79.5	75.6	75.3	74.0
Naive-Async	71.3	52.7	47.1	39.8
VLASH	76.0	71.7	71.8	70.2

Source: own work

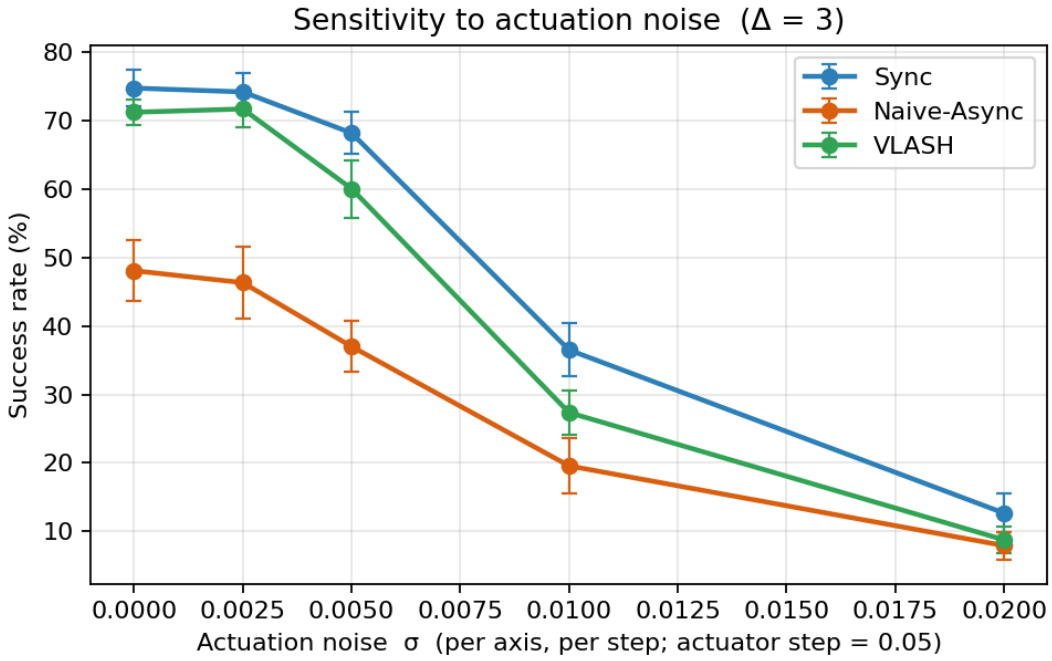
Synchronous inference holds near 75% across the range, and VLASH stays within a few points of it, from 76.0% down to 70.2%. Naive asynchronous inference falls away sharply, from 71.3% at the smallest jump to 39.8% at the largest, because a larger jump means faster motion during inference and a larger misalignment. The VLASH lead over naive widens from about five points to about thirty as the target grows more dynamic, which matches the large-scale finding that VLASH helps most on fast, reactive tasks, namely the Kinetix result in Section 3.4.

3.3.6 Sensitivity to actuation noise

Rollforward sums the commanded deltas, so it assumes the robot executes each delta exactly. Real actuators do not. To measure how much of the method survives without that assumption, every executed action in this experiment receives zero-mean Gaussian noise of standard deviation σ per axis, while the planner and the rollforward still see only the commanded values. The rolled-forward state then misses the accumulated noise of the pending steps, about $\sigma\sqrt{(2\Delta)}$ in norm, and that drift is exactly the part of the future state the method cannot see. The sweep runs at Δ = 3 with σ from 0 to

0.02 against an actuator step of 0.05, so the largest setting corrupts each step by roughly 40% of the largest commanded motion.

Fig. 10. Success rate versus actuation noise σ at Δ = 3. The VLASH margin over naive async survives moderate noise and closes only when the noise itself dominates the motion.



Source: own work

Tab. 6. Success rate (%) versus actuation noise σ (Δ = 3, mean over 50 trials).

σ (per axis, per step)	**0**	**0.0025**	**0.005**	**0.01**	**0.02**
Sync	74.8	74.2	68.2	36.5	12.7
Naive-Async	48.1	46.3	37.0	19.6	7.9
VLASH	71.2	71.7	60.1	27.4	8.7

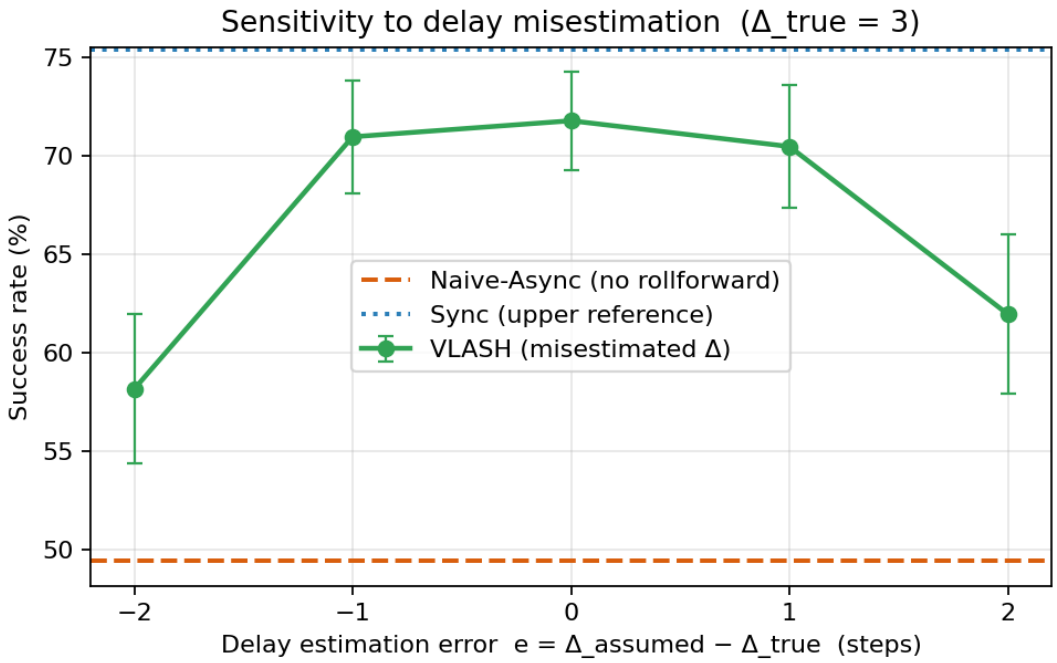
Source: own work

Figure 10 and Table 6 show two regimes. Up to σ = 0.005, one tenth of the actuator step, the ordering and the margins survive: VLASH keeps a lead of 23 to 25 points over naive async and stays within 8 points of the synchronous ceiling. Past that point the noise itself starts to dominate the tracking error. Every controller falls together, and at σ = 0.02 the three curves meet near the floor, with 12.7%, 8.7%, and 7.9% success. The reading is direct: unmodelled noise erodes the advantage only once it grows comparable to the commanded motion, and at that level no inference scheme, synchronous included, holds the tolerance either. A position-controlled arm whose actuation noise sits an order of magnitude below its step size pays almost nothing for the exactness assumption.

3.3.7 Sensitivity to delay misestimation

Rollforward also needs to know how far to roll. The controller estimates the delay Δ before inference finishes, and real inference latency jitters with system load, so the estimate can be off by a step or two. This experiment fixes the true delay at Δ = 3 and lets the controller roll forward Δ + e steps, with the estimation error e swept from −2 to +2. At e = 0 the method is exact; e = −3 would fall all the way back to naive async.

Fig. 11. VLASH success rate under delay estimation error e at Δ_true = 3, against the naive-async and synchronous reference levels. A one-step error costs about one point.



Source: own work

Tab. 7. VLASH success rate (%) under delay estimation error e (Δ_true = 3); naive-async reference 49.4%, synchronous reference 75.4%.

e = Δ_assumed − Δ_true	**−2**	**−1**	**0**	**+1**	**+2**
VLASH success (%)	58.2	71.0	71.8	70.5	62.0
± SD over 50 trials	3.8	2.9	2.5	3.1	4.0

Source: own work

Figure 11 shows a flat-topped curve. A one-step error in either direction costs about one point, 71.0% and 70.5% against 71.8% at the exact estimate, which sits inside the trial-to-trial spread. A two-step error costs more, 58.2% when the controller under-rolls and 62.0% when it over-rolls, yet both still clear the naive baseline of 49.4% by a wide margin. Rollforward degrades gracefully because a partial rollforward removes a proportional share of the misalignment; even a crude latency estimate buys most of the benefit. For deployment this means a moving average of recent inference times is enough to set Δ. The scheduler needs no exact clock.

3.3.8 Cost and reproducibility of the study

The whole study runs on a CPU. One full run, the fifteen delay-and-method settings of Table 4 over 50 trials each plus the robustness, noise, and delay-estimation sweeps of Tables 5 to 7, finishes in well under a minute on an ordinary laptop and writes every figure and CSV used in this chapter. A single base seed fixes all randomness, so re-running the module reproduces every reported number exactly. The code and the generated results are kept under version control, which makes the experiments straightforward to repeat or extend.

3.4 Corroboration with Large-Scale VLASH Results

The large-scale results reported by the VLASH authors [1] show the same trends on a real VLA model and real robots. Table 8 reproduces the published π0.5 numbers across the four LIBERO sub-

benchmarks. At low delay VLASH matches synchronous accuracy, 97.2% at Δ = 1 and 97.1% at Δ = 2 against the 96.8% baseline, with 1.17× and 1.31× speed-ups, and it falls off gently at higher delay, 94.6% at Δ = 3 and 93.1% at Δ = 4, with speed-ups up to 1.47×. One secondary result is worth a look. A model fine-tuned and run with no state input scores a little higher than the state-conditioned model under plain synchronous inference, 97.7% against 96.8%. The base VLA, in other words, barely uses the proprioceptive state, and that is what motivates the temporal-offset augmentation of Section 2.2.2: the augmentation forces the model to read the state, so at deployment it can use the rolled-forward state instead of ignoring it.

The same study also fine-tunes a second, smaller model, SmolVLA [4], with the identical offset augmentation. VLASH carries over to it without any change to the recipe, the first sign that future-state awareness belongs to the training procedure rather than to the π0.5 architecture. Chapter 4 returns to this generalisation question as a direction for further work.

Tab. 8. π0.5 on LIBERO under different inference delays (reported by [1]).

Method	Δ	Spatial	Object	Goal	LIBERO-10	Avg SR (%)	Speedup
Sync	0	97.3	99.6	96.7	93.5	96.8	1.00×
Sync (w/o state)	–	98.5	99.6	97.3	95.4	97.7	1.00×
VLASH	1	98.8	99.2	96.7	94.4	97.2	1.17×
VLASH	2	97.5	99.2	97.3	94.6	97.1	1.31×
VLASH	3	94.4	98.8	93.3	91.9	94.6	1.47×
VLASH	4	92.5	96.9	93.3	89.6	93.1	1.45×

Source: reported by [1]

On the fast-physics Kinetix benchmark, built to stress reaction under delay, the contrast is sharper than on LIBERO. At a four-step delay VLASH reaches 81.7% against 51.2% for naive asynchronous inference, a 30.5-point gain, and it also beats Real-Time Chunking, which pays the extra inpainting cost. This is where the misalignment hurts most, and it mirrors the steep collapse of the naive baseline in the simulation.

On three physical tasks, pick-and-place, stacking, and sorting, on real arms, VLASH scores the highest average at 94%, ahead of naive asynchronous inference at 89.7% and synchronous inference at 83%, and it finishes about 1.12× faster than synchronous control. Synchronous inference scores lowest here because its stalls slow the task under the time-aware scoring. Action quantization pushes the speed-up further: q = 2 reaches 2.03× with no accuracy lost, and q = 3 reaches 2.67× for a 4.7-point drop in score. Asynchronous inference also overlaps computation with execution, which cuts the maximum reaction latency from “finish the current chunk, then infer” to “infer only”. On an RTX 4090 it falls from 536 ms to 36 ms, a 14.9× cut; on a faster RTX 5090 it reaches 17.4×; on a slower RTX 5070 it is still 8.8×. Those gains let VLASH handle genuinely dynamic tasks, such as a human–robot ping-pong rally, that synchronous control cannot.

VLASH also changes the cost of training, not only the cost of inference. Because the shared-observation pass encodes the roughly 700 observation tokens once and reuses them across the offset branches, each training step runs about 3.26 times faster than handling the offsets separately at the same effective batch size [1]. The augmented model converges a little more slowly early on but reaches the same final accuracy, so the speed-up comes at almost no quality cost. At student scale this matters: it brings a full offset-augmented fine-tune within reach of a single GPU.

3.5 Discussion

Why VLASH works. The simulation and the large-scale results point the same way. Asynchronous inference is fast because computation and execution overlap, and that overlap is also what creates the misalignment: the chunk is committed against a state the robot has already left. VLASH removes the misalignment at the source, before the policy runs, by conditioning on the rolled-forward state. RTC and A2C2 patch the chunk after the fact, RTC with runtime inpainting and A2C2 with a per-step correction head; VLASH adds only a vector sum at runtime and touches nothing in the architecture. The simulation pins the mechanism down: ε goes to zero (Figure 4) and the accuracy follows (Figures 5–6).

Agreement across scales. The controlled simulation and the full-scale evaluation agree on every qualitative point: misalignment grows with delay, naive async collapses as the delay rises and worst of all in dynamic settings, VLASH tracks the synchronous ceiling, and a residual gap stays that comes from stale perception rather than from robot-state error. An independent small-scale rebuild lands on the same behaviour as the published numbers, which makes it likely that the behaviour comes from the misalignment mechanism and not from a quirk of one benchmark.

Limitations. Three are worth stating plainly. First, the simulation is a kinematic abstraction with no contact dynamics, no real perception, and an idealised planner, so it confirms the mechanism without predicting absolute success rates. Second, the simulation and VLASH both fix only the robot state; neither predicts the future scene, so a gap to synchronous accuracy stays whenever the environment shifts during inference. Third, the rollforward is exact only for delta actions; a Cartesian or absolute-joint action space would need forward kinematics and would not come for free.

Threats to validity. The success rates in Section 3.3 belong to this reaching task, this staircase target, and the parameters of Section 2.4, so the absolute numbers are not predictions for real manipulation. What carries over is the ordering of the methods and the shape of each curve, and both match the large-scale evaluation. Averaging over 50 trials and reporting the ±1 standard-deviation bars (Figures 5 and 6) keeps that ordering stable rather than a single-seed artefact.

How far the assumptions carry. Sections 3.3.6 and 3.3.7 put numbers on the two assumptions rollforward makes. The exactness assumption holds until actuation noise reaches about a fifth of the commanded step, well above what a position-controlled arm produces, and the known-delay assumption tolerates a one-step estimation error at a cost of about one point of success. Neither assumption is fragile. That backs the plug-and-play claim of Section 1.5 with measurements: the method keeps its margin under the kinds of imperfection a real deployment brings, and it fails only where every inference scheme fails with it.

Practical implications. The mechanism is cheap to adopt. A team already running an action-chunking VLA needs only the delta-action representation, a sum over the pending actions, and a fine-tune with offset augmentation; the inference stack and the model architecture stay the same. That low cost, rather than a new architecture, is what makes future-state awareness attractive for deployment.

What the abstraction keeps and drops. The testbed keeps the three things the argument needs: delta actions, an open-loop chunk, and an injected delay. It drops contact, friction, and real images. The trade is deliberate. With the heavy parts gone, the misalignment is the only moving piece left, so any change in the metrics has a single cause. The price is that the testbed speaks to the relative standing of the methods, not to absolute task difficulty.

Why the residual gap is the right target. In both the simulation and the full-scale results, the only error VLASH does not remove comes from the stale image. That points the way for future work: the next gain will come from predicting the scene, not from a better state estimate, because the state estimate is already exact. A method that closed the visual gap would, in principle, lift VLASH all the way to the synchronous ceiling.

3.6 Summary of the Chapter

This chapter brought two lines of evidence together. The simulation testbed showed, under controlled and reproducible conditions, that prediction–execution misalignment grows with delay for naive asynchronous inference, that state rollforward removes it, and that the accuracy and smoothness of VLASH stay near the synchronous ceiling while the naive baseline collapses; at $\Delta = 4$ the success rates are 73.8% for synchronous, 69.5% for VLASH, and 35.4% for naive. Two further sweeps stressed the method's own assumptions: the VLASH margin survives actuation noise up to a tenth of the actuator step, and a one-step error in the delay estimate costs about one point of success. The large-scale evaluation showed the same trends on $\pi 0.5$: near-synchronous accuracy with 1.17–1.47× speed-ups on LIBERO, a 30.5-point lead over naive async on the dynamic Kinetix benchmark, the top score on three real-world tasks, and reaction-latency cuts up to 17.4×. A small residual gap appears in both settings and traces back to the stale visual observation, the main limitation left and a target for future work.

CHAPTER 4: Conclusion and Future Work

4.0 Conclusion

This thesis took on a core bottleneck in deploying large Vision-Language-Action models for real-time control: the prediction–execution misalignment that appears once asynchronous inference removes the action stall. That misalignment is what makes naive asynchronous methods jitter and miss, and the thesis showed it can be removed by a simple, cost-free step, conditioning the policy on a future robot state summed from the pending actions, which is state rollforward.

VLAs such as π0.5 need tens to hundreds of milliseconds per inference, which opens a one-to-four-step gap between the moment an observation is captured and the moment the actions run. Synchronous inference avoids the misalignment but stalls the robot on every query. Naive asynchronous inference drops the stall and leaves the misalignment. RTC and A2C2 fix the misalignment but pay with runtime work or a trained add-on. VLASH handles all of this with state rollforward, temporal-offset augmentation, and the shared-observation training trick, adding nothing at deployment beyond a single vector sum.

Assessment of the objective. The thesis met the objectives set in the Introduction. It reviewed VLA models and real-time inference, analysed the misalignment, explained the future-state-aware method, and built an original simulation testbed that compares the three mechanisms. The testbed gave quantified, reproducible evidence: misalignment for naive asynchronous inference grows with delay, to $\varepsilon = 0.036$ at $\Delta = 4$, and rollforward removes it, $\varepsilon = 0$, while VLASH holds 69.5% success at $\Delta = 4$ against 73.8% for synchronous and 35.4% for naive. These small-scale results match the full-scale VLASH trends, near-synchronous accuracy with 1.17–1.47× speed-ups on LIBERO, a 30.5-point lead over naive async on Kinetix, and reaction-latency cuts up to 17.4×, so the objective was met to a high degree.

Comparison with existing solutions. Against naive asynchronous inference, the approach restores accuracy and keeps the latency advantage. Against RTC it drops the runtime inpainting cost. Against A2C2 it needs no correction module and no architectural change. The main weakness of the author's own contribution is the simulation itself: a kinematic abstraction with no contact dynamics and no real perception, it confirms the mechanism rather than predicting absolute task performance, and its residual VLASH-versus-synchronous gap, caused by stale perception, repeats the limitation the full method also carries.

Broader significance. Future-state awareness turns a deployment problem into a training recipe. Instead of reworking the inference stack for each new model, a team folds offset augmentation into the usual fine-tune and reads a rolled-forward state at run time. As VLAs move toward faster, more reactive tasks, that recipe keeps the accuracy of synchronous control at the speed of asynchronous control, which is the combination that dynamic manipulation has lacked.

4.1 Future Work

Several directions stay open.

The sharpest limitation is that rollforward fixes the robot state but not the visual scene. When an object moves during inference, the policy still works from a stale image. A natural fix is to predict the future observation with a learned world model or a small video-prediction network, then condition the policy on both the rolled-forward state and the predicted image. The simulation in this thesis

already isolates this gap, since its residual VLASH-versus-synchronous difference comes entirely from the stale target, so it would make a cheap testbed for a visual-rollforward prototype before any robot work.

A second direction tests the method beyond π0.5 and SmolVLA. Newer architectures such as GR00T N1 [15], Gemini Robotics [17], and OpenVLA [16] differ in backbone and action head, and confirming that offset augmentation transfers to them would show that future-state awareness is a property of the training recipe rather than of one model.

A third direction is the deployment this thesis was scoped around: a real fine-tune on the author's RTX 3050 (6 GB). The plan is concrete. Load π0.5 in 4-bit with QLoRA, attach LoRA adapters to the attention projections, and keep the state and action projection layers fully trainable, since those carry the future-state signal. Fine-tune on a small LIBERO subset with offset augmentation and the shared-observation pass to stay inside the memory budget, then evaluate synchronous, naive asynchronous, and VLASH inference at delays of one to four steps. Combining this with quantization-aware training and distillation would push the same idea onto smaller embedded hardware.

Two smaller refinements follow. An adaptive scheduler could set the inference launch point from the measured latency instead of a fixed offset, and a learned, phase-dependent quantization policy could use coarse macro-actions in free space and fine actions during contact.

Finally, pairing reactive inference with embodied memory such as Multi-Scale Embodied Memory [18] points toward robots that react fast and still hold a coherent plan over a long task, which is the combination that household-scale manipulation will need.

Bibliography

[1] J. Tang, Y. Sun, Y. Zhao, S. Yang, Y. Lin, Z. Zhang, J. Hou, Y. Lu, Z. Liu, and S. Han, "VLASH: Real-Time VLAs via Future-State-Aware Asynchronous Inference," arXiv:2512.01031, 2025.

[2] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, et al., "π0: A Vision-Language-Action Flow Model for General Robot Control," arXiv:2410.24164, 2024.

[3] Physical Intelligence, K. Black, et al., "π0.5: a Vision-Language-Action Model with Open-World Generalization," arXiv:2504.16054, 2025.

[4] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, et al., "SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics," arXiv:2506.01844, 2025.

[5] K. Black, M. Y. Galliker, and S. Levine, "Real-Time Execution of Action Chunking Flow Policies," arXiv:2506.07339; NeurIPS, 2025.

[6] "Leave No Observation Behind: Real-Time Correction for VLA Action Chunks (A2C2)," arXiv:2509.23224, 2025.

[7] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.

[8] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (ACT)," in Robotics: Science and Systems (RSS), 2023.

[9] M. Matthews, M. Beukman, F. Christianos, L. Schäfer, M. Foster, et al., "Kinetix: Investigating the Training of General Agents through Open-Ended Physics-Based Control Tasks," in International Conference on Learning Representations (ICLR), 2025.

[10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," in International Conference on Learning Representations (ICLR), 2022.

[11] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow Matching for Generative Modeling," in International Conference on Learning Representations (ICLR), 2023.

[12] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, et al., "RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control," in Conference on Robot Learning (CoRL), 2023.

[13] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, et al., "PaliGemma: A Versatile 3B VLM for Transfer," arXiv:2407.07519, 2024.

[14] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid Loss for Language Image Pre-Training (SigLIP)," in IEEE/CVF International Conference on Computer Vision (ICCV), 2023.

[15] NVIDIA, "GR00T N1: An Open Foundation Model for Generalist Humanoid Robots," arXiv:2503.14734, 2025.

[16] M. J. Kim, K. Pertsch, S. Karamcheti, et al., "OpenVLA: An Open-Source Vision-Language-Action Model," arXiv:2406.09246, 2024.

[17] A. Abdolmaleki, S. Abeyruwan, J. Ainslie, et al., "Gemini Robotics 1.5: Pushing the Frontier of Generalist Robots," arXiv:2510.03342, 2025.

[18] M. Torne, K. Pertsch, H. Walke, K. Vedder, S. Nair, B. Ichter, et al., "Multi-Scale Embodied Memory (MEM): VLAs with Long and Short-Term Memory," Physical Intelligence, 2026.

[19] R. Cadene, S. Alibert, A. Soare, Q. Gallouedec, et al., "LeRobot: State-of-the-Art Machine Learning for Real-World Robotics in PyTorch," github.com/huggingface/lerobot, 2024.

List of Figures

List of Tables

Summary

This thesis studies real-time inference for Vision-Language-Action (VLA) models in reactive robotics. Action-chunking VLAs such as π0.5 are accurate but slow to deploy. Synchronous inference stalls the robot on every model query, while naive asynchronous inference drops the stall and brings in a prediction–execution misalignment, because the policy conditions on a state the robot has already left by the time its actions run. The thesis analyses this misalignment and studies the VLASH method, which makes the policy future-state-aware by rolling the robot state forward through the already-known pending actions, an exact, zero-overhead vector sum that the delta-action representation allows.

As its practical contribution, the thesis designs and builds an original, fully reproducible simulation testbed that compares synchronous, naive asynchronous, and future-state-aware inference on a reactive reaching task while sweeping the inference delay. The experiments show that misalignment grows with delay for naive asynchronous inference and that state rollforward removes it, that VLASH keeps task accuracy and motion smoothness near the synchronous ceiling while the naive baseline collapses, and that a small residual gap traces back to stale visual perception. These findings reproduce, at small scale, the trends reported for VLASH at full scale. The thesis closes with future work, including visual-observation rollforward and deployment on resource-constrained hardware.

Keywords: Vision-Language-Action models, asynchronous inference, real-time robotics, state rollforward, action chunking, π0.5, VLASH.